

Unidad III: Mapeo sobre múltiples argumentos y otros patrones de iteración

Denis Javier Rivera Arbaiza RA21117

Contents

Introducción	2
1. Mapeo sobre múltiples argumentos	2
1.1 La función <code>map2()</code>	3
1.2 Ejemplo con una función personalizada	4
1.3 <code>map2()</code> con columnas de una tabla	5
1.4 La función <code>pmap()</code>	5
1.5 <code>pmap()</code> con una función personalizada	7
1.6 Diferencia entre <code>map()</code> , <code>map2()</code> y <code>pmap()</code>	8
2. Otros patrones de bucles for	9
2.1 La función <code>walk()</code>	9
2.2 Ejemplo con nombres	10
2.3 <code>keep()</code> y <code>discard()</code>	10
2.4 <code>some()</code> y <code>every()</code>	11
2.5 <code>detect()</code> y <code>detect_index()</code>	12
2.6 <code>head_while()</code> y <code>tail_while()</code>	13
2.7 La función <code>reduce()</code>	14
2.8 Reducir varias tablas	14
2.9 La función <code>accumulate()</code>	15
3. Ejercicios	16
Ejercicio 1	16
Ejercicio 2	16
Ejercicio 3	16
Ejercicio 4	17
Ejercicio 5	17
Ejercicio 6	17

Ejercicio 7	18
Ejercicio 8	18
Ejercicio 9	18
Ejercicio 10	19

```
library(purrr)
library(dplyr)
library(tibble)
```

Introducción

En los temas anteriores estudiamos cómo usar las funciones `map()` para aplicar una misma función a muchos elementos.

Por ejemplo:

```
map_dbl(mtcars, mean)
```

```
##      mpg      cyl    disp      hp      drat      wt      qsec
## 20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
##      vs      am     gear     carb
##  0.437500  0.406250  3.687500  2.812500
```

Ese código aplica `mean()` a cada columna de `mtcars`.

Sin embargo, en algunos problemas necesitamos iterar sobre **más de un argumento al mismo tiempo**.

Por ejemplo, podríamos querer generar números aleatorios usando diferentes medias y diferentes desviaciones estándar.

También existen otros patrones de iteración útiles, como:

- aplicar una función por sus efectos secundarios;
- conservar o eliminar elementos según una condición;
- verificar si alguna o todas las entradas cumplen cierta condición;
- reducir una lista a un solo objeto.

En este documento estudiaremos los últimos dos temas de la unidad:

1. Mapeo sobre múltiples argumentos.
2. Otros patrones de bucles `for`.

1. Mapeo sobre múltiples argumentos

Hasta ahora hemos usado `map()` para recorrer un solo objeto.

Por ejemplo:

```
medias <- list(5, 10, -3)

map(medias, rnorm, n = 5)
```

```
## [[1]]
## [1] 3.725980 2.461599 3.304368 4.349922 4.426302
##
## [[2]]
## [1] 9.931161 10.114054 10.123845 9.828544 10.650086
##
## [[3]]
## [1] -3.761099 -2.078083 -2.681487 -5.268099 -3.756304
```

En este caso, `map()` toma cada valor de `medias` y lo usa como argumento de `rnorm()`.

El código anterior es parecido a escribir:

```
rnorm(n = 5, mean = 5)
rnorm(n = 5, mean = 10)
rnorm(n = 5, mean = -3)
```

Aquí estamos variando un solo argumento: la media.

Pero a veces queremos variar dos o más argumentos al mismo tiempo.

1.1 La función `map2()`

La función `map2()` sirve para iterar sobre dos objetos en paralelo.

Su estructura general es:

```
map2(.x, .y, .f)
```

donde:

- `.x` es el primer objeto que queremos recorrer;
- `.y` es el segundo objeto que queremos recorrer;
- `.f` es la función que queremos aplicar.

Por ejemplo, supongamos que queremos generar muestras normales con diferentes medias y diferentes desviaciones estándar.

```
medias <- list(5, 10, -3)
desviaciones <- list(1, 5, 10)

simulaciones <- map2(
  medias,
  desviaciones,
  rnorm,
  n = 5
)

simulaciones
```

```
## [[1]]
## [1] 5.699083 4.310703 6.669413 4.308762 4.247074
##
## [[2]]
## [1] 6.697887 9.440196 8.818716 11.929770 9.216049
##
## [[3]]
## [1] 0.05354782 3.60451350 13.02402889 -9.79211219 8.00380082
```

Aquí ocurre lo siguiente:

```
rnorm(n = 5, mean = 5, sd = 1)
rnorm(n = 5, mean = 10, sd = 5)
rnorm(n = 5, mean = -3, sd = 10)
```

Es decir, `map2()` toma el primer valor de `medias` con el primer valor de `desviaciones`, luego el segundo con el segundo, y así sucesivamente.

1.2 Ejemplo con una función personalizada

Supongamos que tenemos dos vectores:

```
x <- c(10, 20, 30)
y <- c(1, 2, 3)
```

Queremos restar elemento a elemento:

```
map2_dbl(x, y, function(a, b) {
  a - b
})
```

```
## [1] 9 18 27
```

También podemos escribirlo con la sintaxis abreviada:

```
map2_dbl(x, y, ~ .x - .y)
```

```
## [1] 9 18 27
```

En esta sintaxis:

- `.x` representa el elemento actual del primer vector;
- `.y` representa el elemento actual del segundo vector.

Por lo tanto:

```
map2_dbl(x, y, ~ .x - .y)
```

```
## [1] 9 18 27
```

equivale a:

```
10 - 1
20 - 2
30 - 3
```

1.3 map2() con columnas de una tabla

También podemos usar `map2()` dentro de un `tibble`.

Supongamos que tenemos información de varios grupos:

```
grupos <- tibble(
  grupo = c("A", "B", "C"),
  minimo = c(0, 10, 100),
  maximo = c(1, 20, 200)
)

grupos
```

```
## # A tibble: 3 x 3
##   grupo minimo maximo
##   <chr>   <dbl> <dbl>
## 1 A         0     1
## 2 B        10    20
## 3 C       100   200
```

Queremos calcular la diferencia entre `maximo` y `minimo`.

```
grupos <- grupos %>%
  mutate(
    amplitud = map2_dbl(maximo, minimo, ~ .x - .y)
  )

grupos
```

```
## # A tibble: 3 x 4
##   grupo minimo maximo amplitud
##   <chr>   <dbl> <dbl>   <dbl>
## 1 A         0     1         1
## 2 B        10    20        10
## 3 C       100   200       100
```

Aquí:

- `.x` representa cada valor de `maximo`;
- `.y` representa cada valor de `minimo`;
- `.x - .y` calcula la diferencia.

1.4 La función `pmap()`

La función `map2()` funciona bien cuando tenemos dos argumentos que cambian.

Pero si tenemos tres o más argumentos que cambian, usamos `pmap()`.

La función `pmap()` recibe una lista de argumentos.

Por ejemplo, supongamos que queremos generar muestras normales variando:

- el tamaño de muestra `n`;
- la media `mean`;
- la desviación estándar `sd`.

```
parametros <- tibble(  
  n = c(5, 5, 5),  
  mean = c(0, 10, 100),  
  sd = c(1, 2, 10)  
)
```

```
parametros
```

```
## # A tibble: 3 x 3  
##       n mean  sd  
##   <dbl> <dbl> <dbl>  
## 1     5     0    1  
## 2     5    10    2  
## 3     5   100   10
```

Podemos usar `pmap()`:

```
simulaciones <- parametros %>%  
  pmap(rnorm)
```

```
simulaciones
```

```
## [[1]]  
## [1]  0.006768121 -0.083304655  1.183592734 -0.420335397  0.683446110  
##  
## [[2]]  
## [1]  9.103176  9.715014  9.626616  7.620316  9.401588  
##  
## [[3]]  
## [1]  79.68048  94.28449  93.42956 100.34139 104.18964
```

Esto genera llamadas equivalentes a:

```
rnorm(n = 5, mean = 0, sd = 1)  
rnorm(n = 5, mean = 10, sd = 2)  
rnorm(n = 5, mean = 100, sd = 10)
```

Una ventaja importante de usar un `tibble` es que los argumentos tienen nombre. Por eso `pmap()` sabe que la columna `n` corresponde al argumento `n`, la columna `mean` al argumento `mean` y la columna `sd` al argumento `sd`.

1.5 pmap() con una función personalizada

También podemos usar `pmap()` con una función creada por nosotros.

Supongamos que tenemos una tabla con datos de productos:

```
productos <- tibble(
  producto = c("A", "B", "C"),
  precio = c(10, 25, 40),
  cantidad = c(3, 2, 5),
  descuento = c(0.1, 0.2, 0.15)
)

productos
```

```
## # A tibble: 3 x 4
##   producto precio cantidad descuento
##   <chr>     <dbl>   <dbl>     <dbl>
## 1 A         10         3         0.1
## 2 B         25         2         0.2
## 3 C         40         5         0.15
```

Queremos calcular el total después del descuento:

$$\text{total} = \text{precio} \times \text{cantidad} \times (1 - \text{descuento})$$

Usamos `pmap_dbl()`:

```
productos <- productos %>%
  mutate(
    total = pmap_dbl(
      list(precio, cantidad, descuento),
      function(precio, cantidad, descuento) {
        precio * cantidad * (1 - descuento)
      }
    )
  )

productos
```

```
## # A tibble: 3 x 5
##   producto precio cantidad descuento total
##   <chr>     <dbl>   <dbl>     <dbl> <dbl>
## 1 A         10         3         0.1    27
## 2 B         25         2         0.2    40
## 3 C         40         5         0.15   170
```

También se puede escribir con la sintaxis abreviada:

```
productos <- tibble(
  producto = c("A", "B", "C"),
  precio = c(10, 25, 40),
```

```

    cantidad = c(3, 2, 5),
    descuento = c(0.1, 0.2, 0.15)
  )

productos <- productos %>%
  mutate(
    total = pmap_dbl(
      list(precio, cantidad, descuento),
      ~ ..1 * ..2 * (1 - ..3)
    )
  )

productos

```

```

## # A tibble: 3 x 5
##   producto precio cantidad descuento total
##   <chr>      <dbl>    <dbl>    <dbl> <dbl>
## 1 A          10         3      0.1     27
## 2 B          25         2      0.2     40
## 3 C          40         5     0.15    170

```

En esta sintaxis:

- ..1 representa el primer argumento;
- ..2 representa el segundo argumento;
- ..3 representa el tercer argumento.

Sin embargo, para estudiantes suele ser más claro usar la versión con `function()`.

1.6 Diferencia entre `map()`, `map2()` y `pmap()`

Podemos resumirlo así:

Función	¿Cuántas entradas recorreremos?	Ejemplo
<code>map()</code>	Una entrada	aplicar <code>mean()</code> a cada columna
<code>map2()</code>	Dos entradas en paralelo	combinar media y desviación
<code>pmap()</code>	Tres o más entradas	usar <code>n</code> , <code>mean</code> y <code>sd</code> juntos

Ejemplo con `map()`:

```
map_dbl(mtcars, mean)
```

```

##      mpg      cyl    disp      hp      drat      wt      qsec
## 20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
##      vs      am     gear     carb
##  0.437500  0.406250  3.687500  2.812500

```

Ejemplo con `map2()`:

```
map2_dbl(c(10, 20, 30), c(1, 2, 3), ~ .x - .y)
```

```
## [1] 9 18 27
```

Ejemplo con `pmap()`:

```
datos <- tibble(  
  a = c(1, 2, 3),  
  b = c(10, 20, 30),  
  c = c(100, 200, 300)  
)  
  
datos %>%  
  pmap_dbl(function(a, b, c) {  
    a + b + c  
  })
```

```
## [1] 111 222 333
```

2. Otros patrones de bucles for

Además de `map()`, `map2()` y `pmap()`, el paquete `purrr` ofrece otras funciones que representan patrones comunes de iteración.

En esta sección estudiaremos:

- `walk()`;
- `keep()` y `discard()`;
- `some()` y `every()`;
- `detect()` y `detect_index()`;
- `reduce()` y `accumulate()`.

2.1 La función `walk()`

La función `walk()` se parece a `map()`, pero se usa cuando nos interesa la acción que realiza la función, no el objeto que devuelve.

Por ejemplo, si queremos imprimir los elementos de una lista:

```
x <- list(1, "a", 3)  
  
walk(x, print)
```

```
## [1] 1  
## [1] "a"  
## [1] 3
```

Aquí lo importante es que `print()` imprime cada elemento en pantalla.

La diferencia conceptual es:

```
map(x, print)
```

devuelve una lista con resultados.

Mientras que:

```
walk(x, print)
```

se usa principalmente por el efecto secundario de imprimir.

2.2 Ejemplo con nombres

Supongamos que queremos imprimir un mensaje para cada columna de `iris`.

```
walk(names(iris), function(nombre) {  
  print(paste("La columna se llama:", nombre))  
})
```

```
## [1] "La columna se llama: Sepal.Length"  
## [1] "La columna se llama: Sepal.Width"  
## [1] "La columna se llama: Petal.Length"  
## [1] "La columna se llama: Petal.Width"  
## [1] "La columna se llama: Species"
```

Este tipo de operación no busca crear una nueva tabla o vector. Solo busca mostrar mensajes.

Por eso `walk()` es una buena opción.

2.3 `keep()` y `discard()`

Las funciones `keep()` y `discard()` sirven para conservar o eliminar elementos de acuerdo con una condición.

La condición debe ser una función que devuelva `TRUE` o `FALSE`.

Por ejemplo, en `iris`, algunas columnas son numéricas y una columna es factor.

```
map_lgl(iris, is.numeric)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species  
##           TRUE           TRUE           TRUE           TRUE      FALSE
```

Si queremos conservar solo las columnas numéricas, usamos `keep()`:

```
iris_numericas <- iris %>%  
  keep(is.numeric)  
  
head(iris_numericas)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width
## 1         5.1         3.5         1.4         0.2
## 2         4.9         3.0         1.4         0.2
## 3         4.7         3.2         1.3         0.2
## 4         4.6         3.1         1.5         0.2
## 5         5.0         3.6         1.4         0.2
## 6         5.4         3.9         1.7         0.4
```

Si queremos eliminar las columnas numéricas y conservar las no numéricas, usamos `discard()`:

```
iris_no_numericas <- iris %>%
  discard(is.numeric)

head(iris_no_numericas)
```

```
##   Species
## 1  setosa
## 2  setosa
## 3  setosa
## 4  setosa
## 5  setosa
## 6  setosa
```

En resumen:

```
keep(x, condicion)
```

conserva los elementos donde la condición es `TRUE`.

```
discard(x, condicion)
```

elimina los elementos donde la condición es `TRUE`.

2.4 `some()` y `every()`

Las funciones `some()` y `every()` sirven para verificar condiciones sobre una colección de elementos.

`some()` pregunta:

¿Algún elemento cumple la condición?

`every()` pregunta:

¿Todos los elementos cumplen la condición?

Por ejemplo:

```
x <- list(1:5, letters, list(10))
```

Preguntamos si algún elemento es de tipo carácter:

```
some(x, is.character)
```

```
## [1] TRUE
```

Preguntamos si todos los elementos son vectores:

```
every(x, is.vector)
```

```
## [1] TRUE
```

Ahora veamos un ejemplo con `iris`.

```
some(iris, is.factor)
```

```
## [1] TRUE
```

Esto pregunta si alguna columna de `iris` es factor.

```
every(iris, is.numeric)
```

```
## [1] FALSE
```

Esto pregunta si todas las columnas de `iris` son numéricas.

Como `Species` no es numérica, el resultado es `FALSE`.

2.5 `detect()` y `detect_index()`

La función `detect()` encuentra el primer elemento que cumple una condición.

La función `detect_index()` devuelve la posición del primer elemento que cumple una condición.

Por ejemplo:

```
valores <- c(3, 5, 8, 2, 10)
```

```
detect(valores, ~ . > 6)
```

```
## [1] 8
```

El primer valor mayor que 6 es 8.

Ahora busquemos su posición:

```
detect_index(valores, ~ . > 6)
```

```
## [1] 3
```

La posición de 8 es 3.

También podemos usarlo con una lista.

```
lista <- list(  
  a = c("uno", "dos"),  
  b = c(1, 2, 3),  
  c = c(TRUE, FALSE)  
)  
  
detect(lista, is.numeric)
```

```
## [1] 1 2 3
```

Esto devuelve el primer elemento numérico.

```
detect_index(lista, is.numeric)
```

```
## [1] 2
```

Esto devuelve la posición del primer elemento numérico.

2.6 head_while() y tail_while()

Las funciones `head_while()` y `tail_while()` conservan elementos desde el inicio o desde el final mientras se cumpla una condición.

Por ejemplo:

```
valores <- c(8, 7, 6, 3, 9, 10)  
  
head_while(valores, ~ . > 5)
```

```
## [1] 8 7 6
```

Este código toma valores desde el inicio mientras sean mayores que 5.

El resultado es:

```
8, 7, 6
```

Se detiene cuando encuentra 3, porque $3 > 5$ es falso.

Ahora desde el final:

```
tail_while(valores, ~ . > 5)
```

```
## [1] 9 10
```

Este código toma valores desde el final mientras sean mayores que 5.

En este caso toma 9 y 10.

2.7 La función `reduce()`

La función `reduce()` sirve para combinar muchos elementos hasta obtener uno solo.

Por ejemplo, supongamos que tenemos varios vectores y queremos encontrar los elementos comunes a todos.

```
vectores <- list(
  c(1, 3, 5, 6, 10),
  c(1, 2, 3, 7, 8, 10),
  c(1, 2, 3, 4, 8, 9, 10)
)

reduce(vectores, intersect)
```

```
## [1] 1 3 10
```

Aquí `intersect()` se aplica repetidamente hasta encontrar la intersección común.

La idea es parecida a:

```
intersect(
  intersect(vectores[[1]], vectores[[2]]),
  vectores[[3]]
)
```

Otro ejemplo: sumar elementos de una lista.

```
numeros <- list(1, 2, 3, 4, 5)

reduce(numeros, `+`)
```

```
## [1] 15
```

Esto equivale a:

```
((((1 + 2) + 3) + 4) + 5)
```

2.8 Reducir varias tablas

Una aplicación muy útil de `reduce()` ocurre cuando tenemos varias tablas y queremos unir las.

Por ejemplo:

```
tabla_edad <- tibble(
  nombre = c("Ana", "Luis"),
  edad = c(20, 25)
)

tabla_notas <- tibble(
  nombre = c("Ana", "Luis"),
  nota = c(8.5, 9.2)
)
```

```

tabla_grupo <- tibble(
  nombre = c("Ana", "Luis"),
  grupo = c("A", "B")
)

tablas <- list(tabla_edad, tabla_notas, tabla_grupo)

reduce(tablas, full_join, by = "nombre")

```

```

## # A tibble: 2 x 4
##   nombre edad  nota grupo
##   <chr> <dbl> <dbl> <chr>
## 1 Ana      20    8.5 A
## 2 Luis     25    9.2 B

```

Aquí `reduce()` aplica `full_join()` repetidamente hasta combinar todas las tablas en una sola.

2.9 La función `accumulate()`

La función `accumulate()` es parecida a `reduce()`, pero conserva los resultados intermedios.

Por ejemplo:

```

numeros <- c(1, 2, 3, 4, 5)

accumulate(numeros, `+`)

```

```
## [1] 1 3 6 10 15
```

El resultado es la suma acumulada:

```

1
1 + 2
1 + 2 + 3
1 + 2 + 3 + 4
1 + 2 + 3 + 4 + 5

```

Es decir:

```
1, 3, 6, 10, 15
```

También podemos usarlo con multiplicación:

```
accumulate(numeros, `*`)
```

```
## [1] 1 2 6 24 120
```

Esto devuelve el producto acumulado.

3. Ejercicios

Ejercicio 1

Usa `map2_dbl()` para sumar elemento a elemento los siguientes vectores:

```
x <- c(10, 20, 30, 40)
y <- c(1, 2, 3, 4)

suma <- map2_dbl(x,y, `+`)
suma
```

```
## [1] 11 22 33 44
```

Ejercicio 2

Usa `map2_dbl()` para calcular el área de varios rectángulos. Usa los siguientes vectores:

```
base <- c(2, 5, 10, 4)
altura <- c(3, 2, 6, 8)

area <- map2_dbl(base,altura, ~ .x*.y)
area
```

```
## [1] 6 10 60 32
```

La fórmula es:

$$\text{área} = \text{base} \times \text{altura}.$$

Ejercicio 3

Crea un `tibble` con tres columnas: `precio`, `cantidad` y `descuento`.

Luego usa `pmap_dbl()` para calcular el total pagado, usando la fórmula:

$$\text{total} = \text{precio} \times \text{cantidad} \times (1 - \text{descuento}).$$

```
ventas <- tibble(
  precio = c(100, 250, 80, 150),
  cantidad = c(2, 1, 5, 3),
  descuento = c(0.10, 0.20, 0.00, 0.15))

pmap_dbl(ventas,function(precio,cantidad,descuento){
  (precio*cantidad*(1-descuento))
})
```

```
## [1] 180.0 200.0 400.0 382.5
```

Ejercicio 4

Usa `map2()` para generar muestras normales con las siguientes medias y desviaciones estándar:

```
medias <- list(0, 10, 100)
desviaciones <- list(1, 2, 10)
map2(medias,desviaciones,~rnorm(5,.x,.y))
```

```
## [[1]]
## [1] -0.20140834 -1.56270651 -0.07343489 -0.90308672  1.88659497
##
## [[2]]
## [1] 10.560095 10.223045  7.543603 11.924863 12.541658
##
## [[3]]
## [1]  92.86728 120.59863 107.98149 109.44878 103.22132
```

Cada muestra debe tener tamaño 5.

Ejercicio 5

Usa `keep()` para conservar únicamente las columnas numéricas de `iris`.

```
head(keep(iris,is.numeric))
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width
## 1          5.1          3.5          1.4          0.2
## 2          4.9          3.0          1.4          0.2
## 3          4.7          3.2          1.3          0.2
## 4          4.6          3.1          1.5          0.2
## 5          5.0          3.6          1.4          0.2
## 6          5.4          3.9          1.7          0.4
```

Ejercicio 6

Usa `discard()` para eliminar las columnas numéricas de `iris`.

```
head(discard(iris,is.numeric))
```

```
## Species
## 1 setosa
## 2 setosa
## 3 setosa
## 4 setosa
## 5 setosa
## 6 setosa
```

Ejercicio 7

Usa `some()` y `every()` para responder:

1. ¿Alguna columna de `iris` es factor?
2. ¿Todas las columnas de `iris` son numéricas?

```
print(paste("¿Alguna columna de iris es factor?", some(iris,is.factor)))
```

```
## [1] "¿Alguna columna de iris es factor? TRUE"
```

```
print(paste("¿Todas las columnas de iris son nuericas?" , every(iris,is.numeric)))
```

```
## [1] "¿Todas las columnas de iris son nuericas? FALSE"
```

Ejercicio 8

Usa `detect()` y `detect_index()` para encontrar el primer valor mayor que 50 en el siguiente vector:

```
valores <- c(10, 20, 45, 60, 80, 5)
print(paste("Primer valor mayor a 50 es ",detect(valores,~ .x > 50)))
```

```
## [1] "Primer valor mayor a 50 es 60"
```

```
print(paste("Posicion del primer valor mayor a 50 es ",detect_index(valores,~ .x > 50)))
```

```
## [1] "Posicion del primer valor mayor a 50 es 4"
```

Ejercicio 9

Usa `reduce()` para encontrar los valores comunes en todos los vectores de la siguiente lista:

```
vectores <- list(  
  c(1, 2, 3, 4, 5),  
  c(2, 3, 5, 7),  
  c(0, 2, 3, 5, 9)  
)  
reduce(vectores, intersect)
```

```
## [1] 2 3 5
```

Ejercicio 10

Usa `accumulate()` para calcular la suma acumulada del siguiente vector:

```
numeros <- c(5, 10, 15, 20)  
accumulate(numeros, `+`)
```

```
## [1] 5 15 30 50
```